1.1 OSI model: data communication

- The OSI model divides communication into seven layers. Data travels down the sender's stack, crosses the medium, and moves up the receiver's stack. Each layer uses the service below it and serves the layer above.

Sender: functions of the seven layers

- 7. Application: provides network services such as HTTP, email and file transfer. Example: a browser creates an HTTP request.
- 6. Presentation: handles data representation, encoding, compression and encryption. UTF-8 and encryption libraries perform such functions in practical systems.
- 5. Session: establishes, manages and ends a dialogue. It may provide synchronization or checkpoints. Internet applications often implement these functions themselves.
- 4. Transport: provides process-to-process delivery using ports. TCP divides a byte stream into segments and uses sequence numbers, ACKs and retransmission for reliable, ordered delivery.
- 3. Network: adds logical addresses and supports routing between networks. Routers forward IP packets toward the destination using routing information.
- 2. Data link: creates frames for one link, using MAC addresses and an error-detection field such as Ethernet FCS. Switches normally forward frames by destination MAC address.
- 1. Physical: represents bits as electrical, optical or radio signals and carries them through the medium.

Encapsulation and an example

- Encapsulation means adding protocol control information as data moves downward. The data from one layer becomes the payload of the next. Ethernet also adds a trailer.

```
Application data
[ TCP header | application data ]
[ IP header | TCP header | application data ]
[ MAC header | IP | TCP | data | FCS ]   ->   bits on the link
```

1.1 OSI model: data communication (cont.)

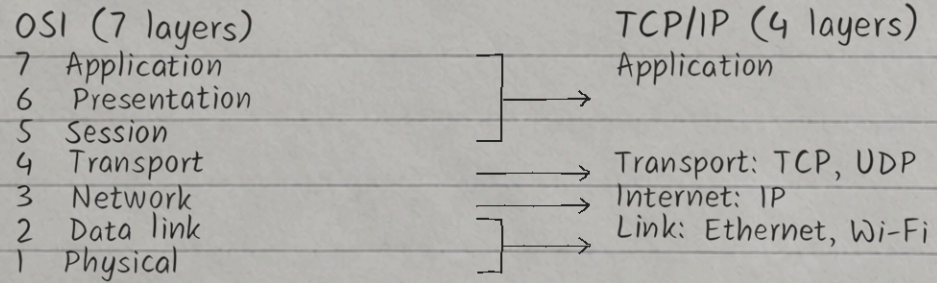
- For an HTTP file request over TCP, TCP adds ports and sequence information; IP adds the endpoint IP addresses; Ethernet adds local-link MAC addresses. The physical interface sends the resulting bits as signals.
- If the server is remote, the first destination MAC belongs to the default gateway. A router removes the incoming frame and creates a new outgoing frame.
- Link addresses change along the route; endpoint IP addresses normally stay the same when there is no NAT.

Receiver: decapsulation

- The receiver recovers bits, checks the Ethernet frame and removes its link header and trailer. IP processes its header and selects TCP. TCP checks, reorders and acknowledges the bytes, removes duplicates, and delivers the stream to the correct application.
- The application interprets the message and displays or stores the file. Thus encapsulation adds control information at the sender, while decapsulation processes and removes it at the receiver.

1.2 OSI and TCP/IP models

- OSI is a seven-layer reference architecture. TCP/IP describes the Internet protocol suite and is commonly shown with four layers. A five-layer teaching version separates physical and data-link functions; the mapping below uses four layers.



1.2 OSI and TCP/IP models (cont.)

Similarities

- Both models divide communication into manageable tasks, use encapsulation, and distinguish applications, transport and routing. Peer entities follow protocols; neighboring layers interact through service interfaces.
- Both support different underlying media. An application can communicate over Ethernet or Wi-Fi without implementing their signaling details. Ports identify application endpoints, IP addresses support inter-network delivery, and MAC addresses serve local links.

Main differences

- Layer count: OSI has seven layers; TCP/IP has four in this convention. TCP/IP combines OSI application, presentation and session functions in its application layer. Its link layer covers the two lowest OSI layers.
- Purpose: OSI supplies a general framework and terminology. TCP/IP is closely associated with protocols used on operational networks. The OSI reference model is distinct from the historical OSI protocol suite.
- Services: OSI explicitly separates services, interfaces and protocols. TCP/IP has broader boundaries; applications often manage their own encoding, encryption and sessions.
- Network layer: OSI can describe connection-oriented or connectionless network services. TCP/IP uses connectionless, best-effort IP. An end-to-end transport such as TCP can add reliability.
- Examples: TCP/IP application protocols include HTTP, DNS and SMTP; transport includes TCP and UDP; the Internet layer includes IP; the link layer includes Ethernet and Wi-Fi access.

1.2 OSI and TCP/IP models (cont.)

Advantages and limitations

OSI	TCP/IP
• Clear division of functions • Modular design; fault isolation • Limits: abstract; some overlap	• Practical interoperability • Open standards; scalable routing • Limits: broader layer boundaries

- Neither model alone guarantees reliability or security. TCP provides reliable ordered bytes; TLS can add authentication and encryption. The mapping is approximate because actual protocols may perform functions associated with several OSI layers.
- OSI is especially useful for explanation and diagnosis, while TCP/IP directly describes the Internet stack in use.

1.3 Troubleshooting with OSI

- First identify who is affected, which service fails, and when the fault began. Check recent changes and compare with a working device. Test one hypothesis at a time, starting with lower layers for a general connectivity failure.

1. Physical

- Inspect power, cables, connectors, link lights and port status. For wireless links, check signal strength and interference.
- Example: a PC reports an unplugged cable. Test a known-good cable and switch port. Restore the link before changing application settings.

2. Data link

- Check VLAN assignment, speed/duplex agreement, switch MAC tables, ARP entries and frame-error counters.
- Example: link lights work, but the PC cannot resolve its gateway's MAC address. Check the VLAN and local switching. High FCS errors can indicate corrupted frames or a link fault.

1.3 Troubleshooting with OSI (cont.)

3. Network

- Verify the IP address, subnet mask, gateway and routes. Test a local host, the gateway and then a remote IP; use ping and traceroute where permitted.
- Example: local printers work but remote servers fail. Check the default gateway, routing and return path. ICMP may be filtered, so a failed ping alone is not proof that the host is down.

4. Transport

- Check the required TCP/UDP port, service listener, firewall rules and connection state. A packet capture can show unanswered SYNs, resets or retransmissions.
- Example: the server answers ping but TCP port 443 fails. Check the HTTPS listener and port filtering; IP reachability does not prove transport-service availability.

5. Session

- Check session establishment, expiry, reconnect behavior and server-side session state. These functions often reside within applications.
- Example: a user connects successfully but is repeatedly logged out. Investigate expired tokens, inconsistent session state or clock settings.

6. Presentation

- Check character encoding, data formats, compression and encryption compatibility.
- Example: TCP connects but TLS reports a certificate error. Verify the hostname, certificate chain and system clock. Unreadable text may instead result from mismatched encodings.

7. Application

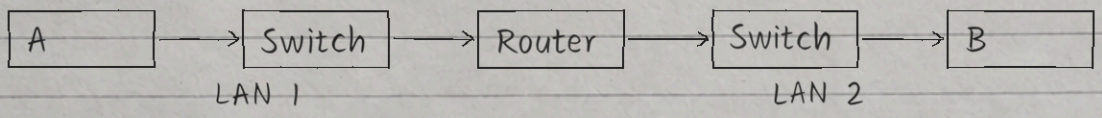
- Check DNS resolution, application configuration, URLs, credentials, permissions and service logs.
- Example: direct IP connectivity works but a hostname fails. Test DNS. An HTTP 403 or 404 response indicates that the service was reached; investigate access rights or the resource.

1.3 Troubleshooting with OSI (cont.)

- After correcting the fault, repeat the original action and test another device or destination. Record the cause and fix. Passing a lower-layer test narrows the search but does not establish that higher-layer services work.

1.4 File transfer across networks

- Assume Ethernet at both ends, a TCP-based file service and no NAT. Let A = 192.168.1.10/24, gateway = 192.168.1.1, and B = 192.168.2.20/24.



- 1. The application opens the file and starts its dialogue. Data is encoded and, if required, encrypted. DNS resolves B's name when a hostname is used.
- 2. TCP establishes a connection, divides the file's byte stream into segments, and adds ports and sequence numbers. ACKs and retransmissions support reliable delivery.
- 3. IP adds A's and B's addresses. A recognizes that B is outside its own subnet, so the next hop is its default gateway.
- 4. A uses ARP to learn the gateway's MAC address. The first frame has source MAC = A and destination MAC = gateway; its IP destination remains B. Bits are sent as signals.
- 5. A switch forwards the frame using its MAC table. The router removes the frame, checks the IP route, reduces TTL and creates a new frame on the outgoing link. On B's LAN, ARP resolves B's MAC address.
- 6. B checks the frame and removes the link and IP headers. TCP orders the bytes, discards duplicates and delivers the stream to the receiving application, which reconstructs the file.

1.4 File transfer across networks (cont.)

- ACKs return toward A. MAC addresses apply to each local hop; the original IP addresses normally remain unchanged across the route. Switches handle local frame delivery, while routers connect networks.

1.5 Layered architecture

- A layer groups related functions and offers a service to the layer above. A protocol defines communication between peers; an interface defines local interaction between adjacent layers.
- Modularity: addressing, transport reliability and physical signaling can be designed separately, reducing complexity.
- Interoperability: agreed formats allow different vendors' devices and software to communicate.
- Independent development: Ethernet can be replaced with another suitable link technology without rewriting the application protocol.
- Reuse: many applications share TCP/IP, and IP can run over different media. Common functions need not be rebuilt for every application.
- Troubleshooting: layered tests separate cable faults, addressing errors and application failures. Clear responsibilities also simplify documentation and testing.
- Limitations: extra headers and processing add overhead. Functions such as error recovery can be duplicated, and strict separation can hide useful information. Layering improves organization but does not automatically guarantee optimum performance.

1.6 TCP: main mechanisms

- TCP is a connection-oriented, full-duplex transport protocol providing a reliable, ordered byte stream over best-effort IP. A connection is identified by endpoint IP addresses and ports. TCP does not preserve application message boundaries.

Connection establishment

- The client sends SYN with sequence number x. The server replies SYN + ACK with its own sequence number y and ACK x + 1. The client returns ACK y + 1. A SYN consumes one sequence number.
- The handshake synchronizes sequence spaces and can negotiate MSS, window scaling and SACK support. It creates endpoint state without reserving a fixed network route.

Reliable transfer

- TCP numbers bytes. A segment starting at 1001 and carrying 500 bytes contains bytes 1001-1500. ACK 1501 means all earlier bytes have arrived in order.
- The checksum detects corruption. The receiver acknowledges in-order progress and can buffer later bytes. The sender retains unacknowledged data and retransmits after a timeout or another loss indication. Sequence numbers identify duplicate data.
- Example: bytes 1001-1500 are lost, but 1501-2000 arrive. The cumulative ACK remains 1001. Once the gap is filled, it can advance to 2001. SACK can report received blocks beyond a gap.
- Timeouts adapt to measured round-trip behavior and back off after repeated failures. TCP can report failure if the path remains unusable; reliability does not guarantee success on a permanently broken connection.

Flow control

- The receiver advertises rwnd, the amount of data it can accept. The sender respects this limit to avoid overflowing the receiver. A zero window pauses ordinary new-data sending; window probes prevent a lost update from causing indefinite waiting.

1.6 TCP: main mechanisms (cont.)

Congestion control

- The sender maintains cwnd to limit traffic entering the network. Slow start, congestion avoidance and loss recovery adjust this limit according to feedback.
  Outstanding data <= min(rwnd, cwnd)
- For rwnd = 32 KB and cwnd = 12 KB, the effective limit is 12 KB. New sending space also subtracts bytes already outstanding. Flow control protects the receiver; congestion control protects the network path.

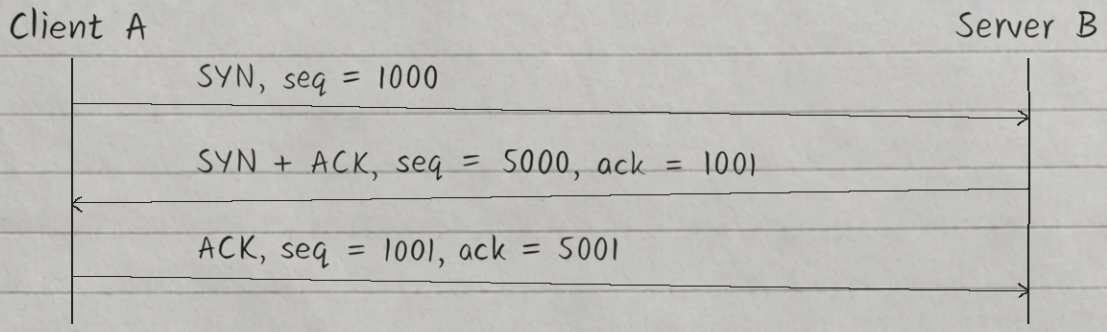
Connection termination

- In a typical close, A sends FIN, B acknowledges it, B later sends FIN, and A acknowledges it. FIN and ACK can sometimes be combined. FIN consumes one sequence number.
- Each direction closes independently, allowing a half-close. The active closer normally enters TIME-WAIT for twice the maximum segment lifetime. RST abruptly aborts the connection. TCP suits file transfer and other applications needing complete, ordered data.

1.7 TCP three-way handshake

- TCP has an independent byte sequence in each direction. Both endpoints must learn the other's initial sequence number and confirm that their own number was received. The three-way handshake performs this synchronization.
- Let the client choose 1000 and the server choose 5000 as illustrative initial sequence numbers. The server initially listens at its service port. Time moves downward in the diagram.

1.7 TCP three-way handshake (cont.)



Step 1: SYN

- The client sends SYN = 1, sequence number 1000, and enters SYN-SENT. This announces its initial sequence number and requests a connection. ACK is not set in the ordinary initial SYN.

Step 2: SYN + ACK

- The server replies with SYN = 1, ACK = 1, sequence number 5000 and acknowledgment 1001. It enters SYN-RECEIVED.
- ACK 1001 confirms the client's SYN because a SYN uses one sequence number. The same segment announces the server's initial number, combining two functions in one message.

Step 3: ACK

- The client returns sequence number 1001 and acknowledgment 5001. The client enters ESTABLISHED; the server does so when this acknowledgment arrives.
- A pure ACK does not consume a sequence number. Therefore the client's first data can still start at 1001. If it sends 200 bytes, those are bytes 1001-1200, and the server acknowledges 1201 after receiving them in order.

Why three steps are needed

- The first message announces the client's sequence number. The second confirms it and announces the server's number. The third confirms that the server's response reached the client.

1.7 TCP three-way handshake (cont.)

- With only two messages, the server would not know whether its SYN + ACK arrived. An old delayed SYN could otherwise create an apparently established connection without a current client wanting it.
- The final ACK ties the exchange to the current sequence numbers and helps prevent old-duplicate confusion. This synchronization is not cryptographic authentication or encryption.

If a message is lost

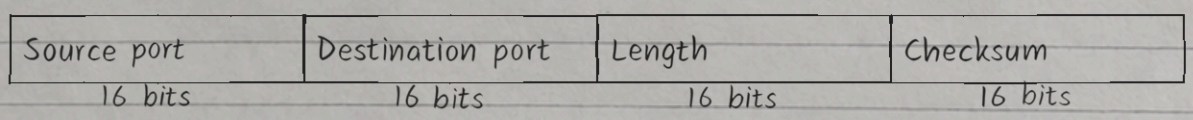
- Lost SYN or SYN + ACK segments are retransmitted after timeout. If the final ACK is lost, the server may repeat SYN + ACK and the client acknowledges it again. A valid ACK carried with later data can also complete establishment.
- Options such as MSS, window scaling and SACK permission can be exchanged in SYN segments. The handshake establishes communication state; it does not reserve bandwidth or a dedicated route.

1.8 TCP and UDP

- Both are transport protocols carried by IP. Both use ports to identify application endpoints and provide checksums for corruption detection. TCP offers an ordered byte stream; UDP offers individual datagrams.

Headers

- TCP normally has a 20-byte minimum header, extending to 60 bytes with options. Fields include ports, sequence number, acknowledgment, data offset, flags, receive window, checksum and urgent pointer.
- UDP has a fixed 8-byte header containing four 16-bit fields. Its length includes the header and payload.



1.8 TCP and UDP (cont.)

- For 100 bytes of data and IPv4 without options: TCP adds 20 + 20 = 40 bytes, giving 140 bytes before link framing. UDP adds 8 + 20 = 28 bytes, giving 128 bytes. ACKs and options can add further TCP overhead.

Reliability and ordering

- TCP acknowledges bytes, retransmits losses, suppresses duplicates and delivers data in order. It does not preserve application write boundaries, so applications need message framing.
- UDP preserves datagram boundaries but does not guarantee delivery, order or duplicate suppression. Its checksum detects corruption without repairing it. UDP checksums can be omitted in IPv4; they are generally required in IPv6, subject to specific exceptions.

Connection management

- TCP normally uses a three-way handshake and keeps connection state, buffers and timers. FIN/ACK exchanges close it orderly; RST aborts it.
- UDP has no built-in handshake or close exchange. A locally connected UDP socket does not create a TCP-like protocol connection over the network.

Flow and congestion control

- TCP uses rwnd to protect the receiver and cwnd to regulate traffic on the path. These limit outstanding data.
- UDP provides neither control itself. The application or a higher protocol must add appropriate rate control, recovery and buffering where needed.

Delay and uses

- TCP's in-order delivery can delay later bytes while a missing portion is retransmitted. UDP avoids that built-in waiting, which can suit updates that become useless when late. UDP is not automatically faster for every task.

1.8 TCP and UDP (cont.)

- TCP examples: conventional HTTP/1.1 and HTTP/2, SSH, SMTP and many file transfers. UDP examples: many DNS queries, DHCP, real-time media and games. DNS can also use TCP.
- Higher protocols can add reliability and security over UDP. For example, HTTP/3 uses QUIC over UDP. Choose transport according to the application's delivery and timing requirements, rather than assuming every UDP-based application is unreliable.

1.9 TCP congestion control

- Congestion occurs when traffic exceeds link or queue capacity, increasing delay and loss. The sender uses cwnd to limit outstanding data; rwnd separately protects the receiver. The effective limit is min(cwnd, rwnd).
- The following is the classical Reno-style explanation. MSS is the segment payload size; ssthresh is the slow-start threshold. Other deployed algorithms can use different growth rules.

1. Slow Start

- For an illustrative start, let cwnd = 1 MSS and ssthresh = 8 MSS. ACKs for new data increase cwnd by roughly one MSS each, giving approximately exponential growth per RTT:
  cwnd: 1, 2, 4, 8 MSS
- This assumes adequate data and regular ACKs. Delayed ACKs or receiver limits can change the pattern. Actual initial windows may be larger than the teaching value of 1 MSS.

2. Congestion Avoidance

- Near the threshold, growth becomes about one MSS per RTT. A common per-ACK approximation is:
  $cwnd = cwnd + MSS \times MSS / cwnd$
- At 8 MSS, each appropriate ACK adds about 1/8 MSS. The idealized evolution is then 8, 9, 10, 11 MSS over successive RTTs. This probes for spare capacity more cautiously.

1.9 TCP congestion control (cont.)

3. Fast Retransmit

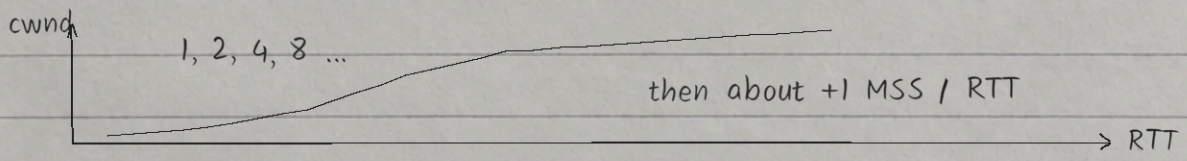
- When later data arrives beyond a missing segment, the receiver repeats its cumulative ACK. In the basic Reno rule, three duplicate ACKs trigger retransmission of the presumed missing segment before timeout.
- One duplicate ACK is insufficient because reordering can also cause it. Loss near the end of a small transfer may still require a timeout if too few later segments generate duplicate ACKs.

4. Fast Recovery

- Set ssthresh = max(flight size / 2, 2 MSS). After the third duplicate ACK, retransmit and temporarily set cwnd = ssthresh + 3 MSS. Further duplicate ACKs allow further temporary increments.
- In the simple single-loss case, an ACK covering the recovered data reduces cwnd to ssthresh; congestion avoidance then continues. Example: flight size 16 MSS gives threshold 8 MSS and temporary cwnd 11 MSS.

Timeout response

- A timeout instead restarts cwnd at 1 MSS under the classical rule, reduces the threshold and resumes slow start. The timeout is backed off. Thus timeout recovery differs from fast recovery.



- Additive increase and multiplicative reduction help control overload. Loss is an imperfect congestion signal, especially on wireless links. ECN can explicitly signal congestion when supported.

1.10 Go-Back-N and Selective Repeat

- ARQ uses error detection, sequence numbers, ACKs and retransmission to recover lost or damaged frames. Both GBN and SR pipeline frames through a sender window instead of waiting after every frame.

Go-Back-N

- The sender permits up to N outstanding frames. The conventional receiver accepts only the next expected frame and discards out-of-order frames; its receive window is 1.
- ACKs are cumulative. If the oldest outstanding frame times out, the sender retransmits it and all later frames already sent but not acknowledged. A conventional implementation uses one oldest-frame timer.

Selective Repeat

- The receiver accepts correct frames within its window, buffers out-of-order frames, and acknowledges them individually or selectively. It delivers buffered data in order once gaps are filled.
- The sender retransmits only frames still needing recovery. It tracks each frame's status and timeout independently, requiring more state and receiver memory.

Example: window 4, frame 2 lost

- Assume successful ACKs and retransmissions. In this example, GBN ACK k acknowledges all frames through k.

Original sends:	1	2 (lost)	3	4
GBN repeats:		2	3	4
SR repeats:		2 only		

- GBN accepts frame 1 but discards 3 and 4 while waiting for 2. It repeats 2, 3 and 4: total = 4 + 3 = 7 sends.
- SR keeps 3 and 4 and retransmits only 2: total = 4 + 1 = 5 sends. Its buffering saves two transmissions here.

1.10 Go-Back-N and Selective Repeat (cont.)

Comparison of the main properties

- Retransmission: GBN repeats a group of outstanding frames; SR repeats only unacknowledged missing or damaged frames.
- Buffering and complexity: GBN needs a simpler receiver. SR stores out-of-order frames and uses more detailed ACK and timer bookkeeping.
- Error handling: both detect errors and retry. Later cumulative ACKs can cover a lost GBN ACK; an SR receiver can re-ACK a duplicate after an ACK is lost.
- Efficiency: both perform well on clean links. SR usually wastes less bandwidth when errors occur, especially with large windows or long propagation delays.

Sequence numbers and window limits

- For m sequence bits, there are 2^m labels. Conventional GBN requires sender window $N <= 2^m - 1$ and receiver window 1.
- For SR with equal sender/receiver windows W, require $2W <= 2^m$, so $W <= 2^{m-1}$. This avoids confusing old duplicates with new frames after wraparound.
- With 3 bits, GBN allows a window of 7 and SR a window of 4. SR with window 5 therefore needs at least 4 sequence bits.

2.1 Link between factory buildings

- Use optical fiber, preferably single-mode fiber with matching optical Ethernet transceivers rated for the required speed and distance. Include cable, splice and connector losses plus a margin in the optical link budget.
- Attenuation: fiber supports kilometer-scale links with low loss. A common twisted-pair Ethernet channel is limited to about 100 m, so a direct 2 km copper run is unsuitable for that case.
- Noise immunity: optical signals are unaffected by electromagnetic interference from motors and power equipment. All-dielectric fiber also electrically isolates the buildings; metallic armor still needs appropriate installation treatment.

2.1 Link between factory buildings (cont.)

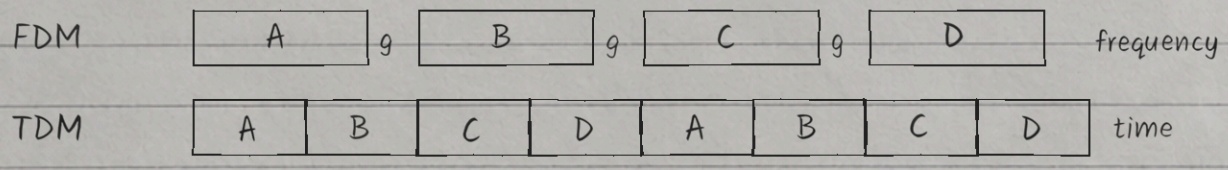
- Bandwidth: suitable fiber and optics support high rates and future upgrades. Actual capacity depends on the equipment and distance specification.
- Cost: optics, termination and installation may cost more initially, but long reach avoids intermediate powered repeaters and improves reliability in the factory environment.
- Simple arrangement: switch A -- optical fiber -- switch B. Fiber is preferable here because the stated copper link suffers both loss and EMI.

2.2 Multiplexing four channels

- Multiplexing combines independent channels on one link; the receiver demultiplexes them. The link needs enough aggregate capacity for all four channels plus overhead.

Frequency-division multiplexing

- FDM assigns each channel a separate frequency band, so all four can transmit simultaneously. Receiver filters separate the bands. Guard bands provide unused gaps to reduce adjacent-channel interference caused by imperfect filtering.



- For channel bandwidth B and three internal guard bands g:
  Total allocation = $4B + 3g$
- For $B = 4$ kHz and $g = 1$ kHz, this gives 19 kHz, excluding any outer-edge allowance. Guard bands improve separation but consume spectrum.

Time-division multiplexing

- For a digital link, assign repeating slots A, B, C, D. Each source uses the link during its own slot. Voice can first be sampled and digitized.

2.2 Multiplexing four channels (cont.)

- Synchronized slots prevent transmissions from overlapping. Guard times may allow for timing uncertainty. These are time gaps, whereas FDM guard bands are frequency gaps.
- Four 64 kbit/s channels require at least 256 kbit/s of payload capacity, plus framing and synchronization overhead. Fixed slots can waste capacity when a source is idle.
- Choose FDM when separate simultaneous frequency bands are suitable, or TDM for coordinated digital transmission. Both methods meet the requirement if bandwidth, timing and overhead are allowed for.

2.3 NRZ, NRZI and Manchester

- H means high level and L means low level. Conventions must be stated because NRZI and Manchester can be described using opposite polarity rules.

NRZ-L

- Choose 1 = H and 0 = L. Each level remains constant for the full bit interval, without returning to zero between bits.
  Levels: H, L, H, H, L, L, H

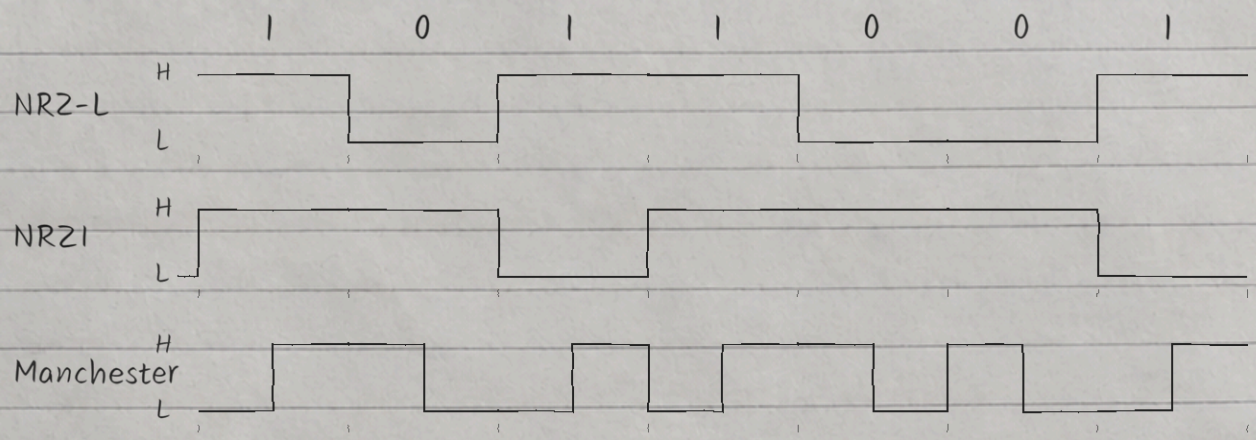
NRZI

- Choose initial level L and a transition at the start of every 1; a 0 leaves the level unchanged.
  Levels: H, H, L, H, H, H, L
- Bit 1 changes L to H; bit 2 stays H; bit 3 changes to L; bit 4 changes to H. Bits 5 and 6 stay H, and bit 7 changes to L.

Manchester

- Use the IEEE 802.3 convention: 1 = low then high; 0 = high then low. Every bit has a transition at its midpoint.
  Half-bit pairs: LH | HL | LH | LH | HL | HL | LH

2.3 NRZ, NRZI and Manchester (cont.)



Each pair of small ticks encloses one bit interval.

Advantages and limitations

- NRZ-L is simple and uses less signaling bandwidth than comparable Manchester coding. Long runs of identical bits give no transitions, making clock recovery difficult. Unbalanced patterns can also cause a DC component or baseline wander.
- NRZI identifies changes relative to the previous level, which helps with overall polarity inversion. Under the stated transition-on-1 rule, long runs of zeros still lack transitions. Additional coding or stuffing may be needed.
- Manchester is self-clocking because every bit has a midpoint transition. With symmetric levels, each bit is balanced. Its cost is two signal elements per bit and a greater bandwidth requirement.
- Extra Manchester transitions can occur at bit boundaries; they are not additional data bits. Decode using the bit timing and midpoint direction.
- For data rate R, a bit lasts $T = 1/R$. Manchester's half-bit interval is $T/2$, giving a half-bit signaling clock of $2R$. The sketches show logical timing only, with time increasing to the right.

2.4 Pure and Slotted ALOHA

- Assume equal packet duration T, the usual independent Poisson attempt model, and a collision channel where overlapping packets fail. G counts attempts, including retries, per packet time; S is successful packets per packet time.
- Pure ALOHA sends whenever a packet is ready. Another start within T before or after it can overlap, so the vulnerable period is 2T. It needs no common slot timing.
- Slotted ALOHA permits starts only at synchronized slot boundaries. The vulnerable period becomes T; collisions occur among devices choosing the same slot.

  Pure: $S = G\, e^{-2G}$

  Slotted: $S = G\, e^{-G}$
- Pure ALOHA reaches its maximum at $G = 1/2$, giving $1/(2e) = 18.4\%$. Slotted ALOHA peaks at $G = 1$, giving $1/e = 36.8\%$. These are ideal maximum normalized throughputs.
- Recommend Slotted ALOHA if the sensors can maintain common timing. Its ideal peak is twice that of pure ALOHA. The key practical assumption is synchronized, equal-duration slots; the numerical results also assume the stated random traffic and collision model.
- Random backoff is still needed. Synchronization consumes resources, and real capture effects or timing errors can change performance.

2.5 Ethernet and Wi-Fi access

- Both use carrier sensing, but nearly simultaneous starts, propagation delay and hidden terminals can still cause collisions.

Traditional shared Ethernet: CSMA/CD

- A station listens before transmitting and monitors the wired medium while sending. If it detects a collision, it stops, issues the collision indication and retries after binary exponential backoff.
- Example: computers connected to an old half-duplex Ethernet hub share one collision domain. CSMA/CD is appropriate there. Modern full-duplex switched Ethernet does not use CSMA/CD.

Wi-Fi: CSMA/CA

- A radio's own transmitted signal overwhelms weaker remote signals, so ordinary Wi-Fi cannot reliably detect a collision while transmitting. Hidden stations may also hear the access point without hearing each other.
- Wi-Fi waits for an idle medium and uses interframe waiting plus random backoff. The counter decreases while idle and freezes while busy. A missing ordinary unicast ACK leads to a retry.
- Optional RTS/CTS and virtual carrier sensing can reduce hidden-terminal collisions, with extra overhead. They do not eliminate all collisions.
- Example: classroom laptops share an access point while some cannot hear each other. Avoidance coordinates access before transmission; ACK feedback identifies unsuccessful delivery afterward.

2.6 CRC calculation

- The generator has 5 bits and degree 4: $g(x) = x^4 + x + 1$. Append four zeros to the data and divide by 10011 using modulo-2 arithmetic. Subtraction is XOR, with no carry or borrow.

  Augmented data = 110101110110000

- Align the generator's leading 1 with each remaining leading 1. The following rows show each XOR operation with leading zeros retained.

Working value	XOR aligned divisor	Result
1 1 0 1 0 1 1 0 1 1 0 0 0 0	1 0 0 1 1 0 0 0 0 0 0 0 0 0	0 1 0 0 1 1 1 0 1 1 0 0 0 0
0 1 0 0 1 1 1 0 1 1 0 0 0 0	0 1 0 0 1 1 0 0 0 0 0 0 0 0	0 0 0 0 0 0 1 0 1 1 0 0 0 0
0 0 0 0 0 0 1 0 1 1 0 0 0 0	0 0 0 0 0 0 1 0 0 1 1 0 0 0	0 0 0 0 0 0 0 0 1 0 1 0 0 0
0 0 0 0 0 0 0 0 1 0 1 0 0 0	0 0 0 0 0 0 0 0 1 0 0 1 1 0	0 0 0 0 0 0 0 0 0 0 0 1 1 1 0

The final four bits are 1110.

- The final remainder has degree below 4, so:

> CRC remainder = 1110
>
> Codeword = 11010110111 1110 = 110101101111110

Receiver check

- Divide the complete received codeword by the same generator, without appending more zeros. The correct codeword gives remainder 0000. A nonzero remainder indicates a detected error.
- Example: changing the final bit gives 110101101111111, whose remainder is 0001, so the error is detected.
- If received data is C + E and C is divisible by the generator, the remainder is that of error pattern E.
- A zero remainder means no error was detected; an error pattern divisible by the generator can escape detection. CRC detects corruption but does not correct it by itself.

2.7 Stop-and-Wait utilization

- Given one-way delay Tp = 20 ms and frame transmission time Tt = 5 ms. Assume negligible ACK transmission and processing times, immediate ACKs and no errors.
- The sender transmits from t = 0 to 5 ms. The final bit arrives at the receiver at 5 + 20 = 25 ms. The ACK returns after another 20 ms, reaching the sender at 45 ms.

Cycle time = Tt + 2Tp = 5 + 40 = 45 ms
U = Tt / (Tt + 2Tp)
= 5/45 = 1/9 = 0.1111 = 11.1%

- Thus the sender transmits for 5 ms and waits about 40 ms per cycle. Payload efficiency is lower if frame overhead is excluded from useful data.
- Check: a = Tp/Tt = 4, so U = 1/(1 + 2a) = 1/9. Both outward and return propagation must be counted.
- If Tp doubles to 40 ms, U = 5/(5 + 80) = 5.88%. Increasing propagation delay reduces utilization for fixed frame time.
- Pipelining can improve this. Under the same ideal timing, a window of about 1 + 2a = 9 frames can keep the sender continuously busy.

2.8 GBN and SR: packet-loss example

- Label packets P1-P15. "Original" means the first attempt for each packet, so the lost first attempts are P4, P8 and P12. Assume all retransmissions succeed and ACKs are never lost. Count data sends only.

Assumptions for an exact GBN count

- Use conventional GBN: receiver window 1, cumulative ACKs, timeout recovery and no fast retransmit. ACKs for in-order packets allow the sender to fill its window before timeout. Finish each outstanding retransmission group and its ACKs before sending new packets.

2.8 GBN and SR: packet-loss example (cont.)

Go-Back-N trace

- 1. Send originals P1-P8. P1-P3 are accepted and acknowledged. P4 is lost; P5-P7 are discarded because the receiver still expects P4. P8's original is also lost. The outstanding window is P4-P8.
- 2. On P4's timeout, resend P4-P8: five successful retransmissions. Cumulative progress now reaches P8. Retransmitted P8 succeeds because the loss rule applies only to originals.
- 3. Send originals P9-P15. P9-P11 succeed; P12 is lost. P13-P15 are discarded while the receiver waits for P12. The outstanding group is P12-P15.
- 4. On timeout, resend P12-P15: four successful retransmissions. All 15 packets are delivered.

Original P1-P8	8 sends
Repeat P4-P8	5 sends
Original P9-P15	7 sends
Repeat P12-P15	4 sends

GBN total = 8 + 5 + 7 + 4 = 24 sends

- Equivalently, 15 originals + 9 retransmissions = 24. The nine retries replace three lost packets and six discarded packets: P5, P6, P7, P13, P14 and P15.

Selective Repeat

- SR buffers correct out-of-order packets within its receive window and acknowledges them selectively. It therefore keeps later packets while waiting for a missing one.
- Only P4, P8 and P12 need one retry each. Once a gap is filled, consecutive buffered packets can be delivered.

SR total = 15 originals + 3 retries = 18 sends

2.8 GBN and SR: packet-loss example (cont.)

- SR saves 24 - 18 = 6 transmissions, or 25% of GBN's sends in this trace. Useful-packet fractions are 15/24 = 62.5% and 15/18 = 83.3%; these are count-based fractions, not elapsed-time throughput.
- GBN's exact count depends on ACK and timeout timing, which the question leaves unspecified. The assumptions above make 24 reproducible. Losing every fourth attempt including retransmissions would be a different problem. SR saves bandwidth at the cost of more buffering and control state.

2.9 Four subnets from a /24

- The original /24 has 8 host bits and 256 addresses. Four subnets require 2 borrowed bits, since $2^2 = 4$. The new prefix is /26, leaving 6 host bits.

  Mask = 255.255.255.192

  Last mask octet = 11000000 (binary)

  Addresses per subnet = $2^6 = 64$

  Usable hosts per subnet = 64 - 2 = 62
- The block increment is 256 - 192 = 64. All entries below share the first three octets 192.168.10.

Subnet	Network	Usable host range	Broadcast
1 (/26)	.0	.1 - .62	.63
2 (/26)	.64	.65 - .126	.127
3 (/26)	.128	.129 - .190	.191
4 (/26)	.192	.193 - .254	.255

All addresses begin with 192.168.10

- The four network addresses are 192.168.10.0/26, 192.168.10.64/26, 192.168.10.128/26 and 192.168.10.192/26. The table abbreviates their common prefix only.

Binary working for the last octet

- Write the last octet as ss hhhhhh. The two s bits select the subnet; the six h bits identify hosts. All-zero host bits give the network, and all-one host bits give the broadcast.

2.9 Four subnets from a /24 (cont.)

00 000000 = 0; 00 111111 = 63

01 000000 = 64; 01 111111 = 127

10 000000 = 128; 10 111111 = 191

11 000000 = 192; 11 111111 = 255

- First usable = network + 1; last usable = broadcast - 1. Four blocks × 64 = 256 addresses, with no gaps or overlap. All four subnet blocks are usable in normal CIDR subnetting.

2.10 Subnet for 50 usable hosts

- For an ordinary IPv4 LAN with h host bits:
  $2^h - 2 >= 50$
- Try $h = 5$: $32 - 2 = 30$ usable addresses. This is /27 and is insufficient. Try $h = 6$: $64 - 2 = 62$ usable addresses, which meets the requirement.

Required prefix = 32 - 6 = /26
Required mask = 255.255.255.192

- Example: 192.168.10.0/26 has network .0, broadcast .63, and usable hosts .1-.62. The network and broadcast addresses cannot be assigned to ordinary hosts.
- A smaller /27 block provides only 30 usable addresses; /28 provides 14. A /25 provides 126 but allocates more than necessary. Thus /26 is the smallest block, or longest suitable prefix.
- After 50 allocations, 12 usable addresses remain. Additional infrastructure, such as a gateway, also uses this pool. The special /31 point-to-point and /32 cases do not apply to a 50-host LAN.